Computación y estructuras discretas I

Unidad 3: Estructuras Discretas no recursivas y análisis de algoritmos – Crecimiento de funciones y Análisis de algoritmos

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI
Material adaptado del material original del profesor Oscar Bedoya

2026

Tabla de contenido

1. Crecimiento de funciones
2. Notación O
3. Notación Ω
4. Notación Θ
5. Terminología de complejidades
6. Clasificación de problemas
7. Análisis de algoritmos

Contenido

1. Crecimiento de funciones

2. Notación O

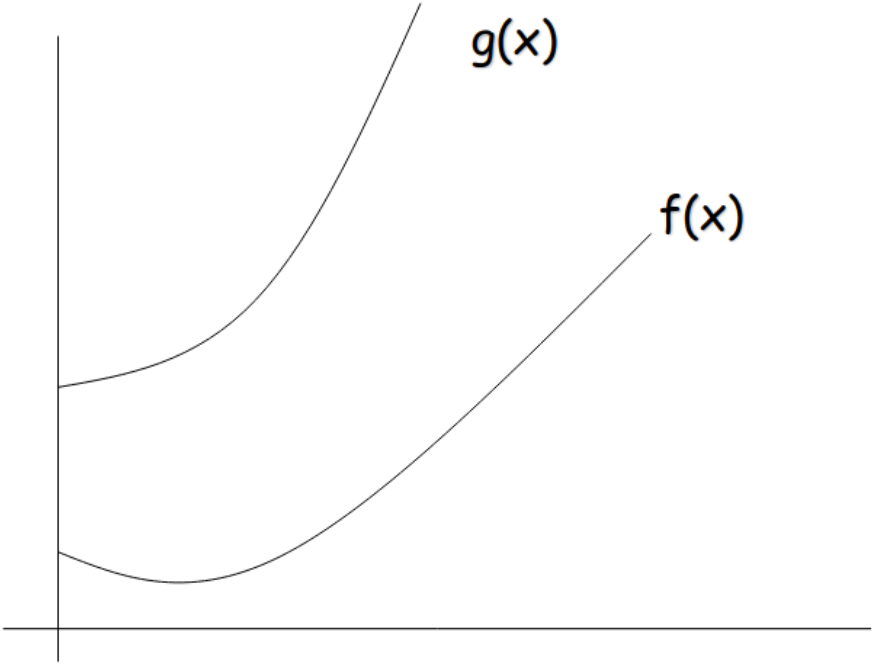
3. Notación Ω

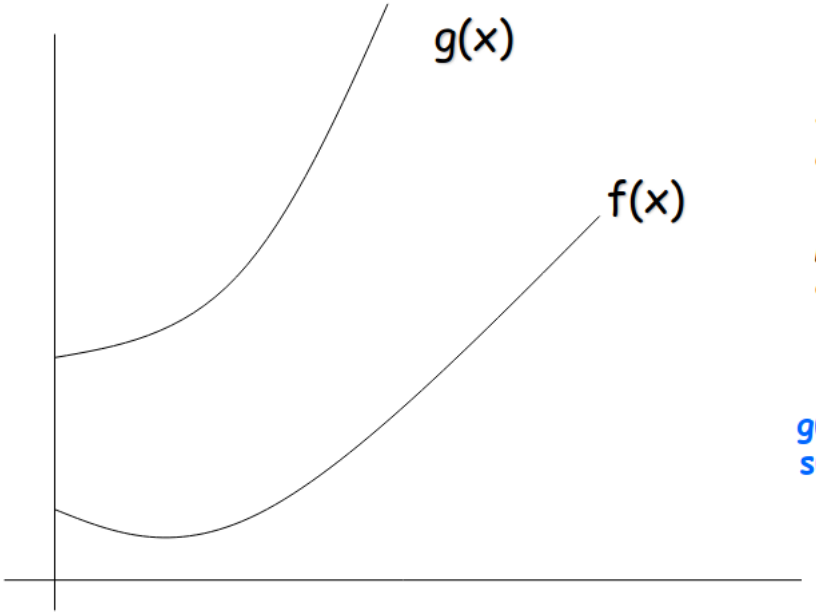
4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

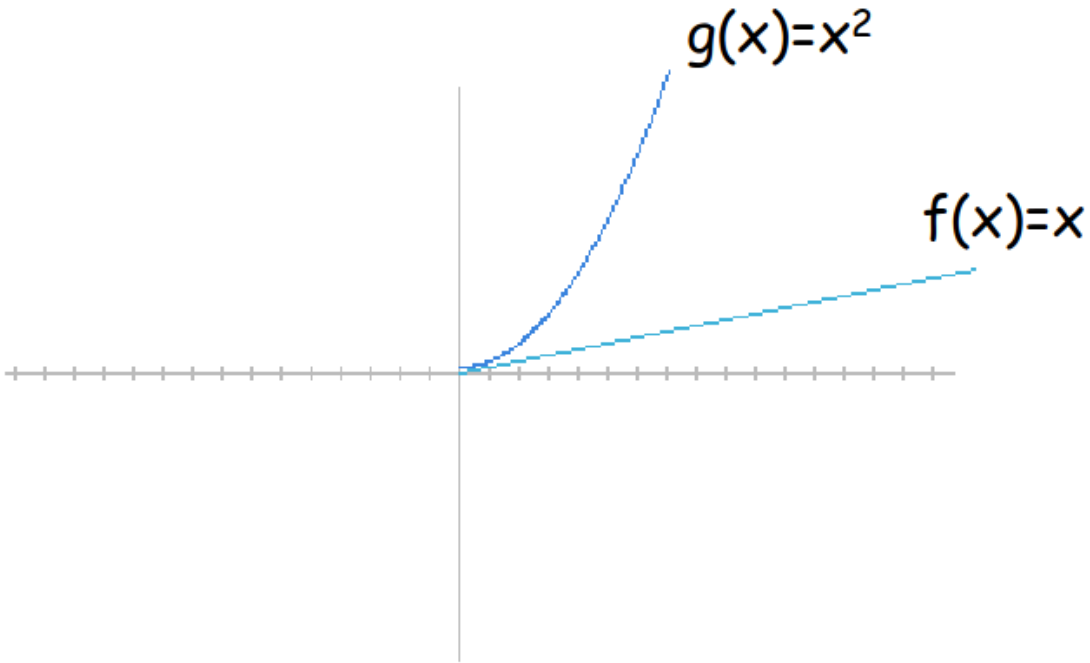
7. Análisis de algoritmos

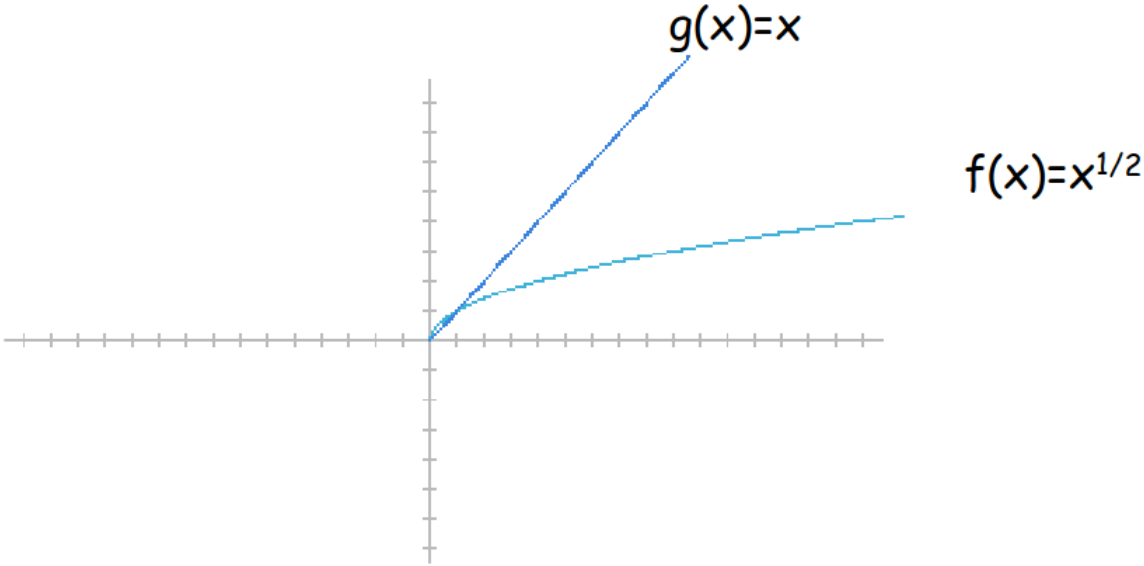

g(x)
f(x)

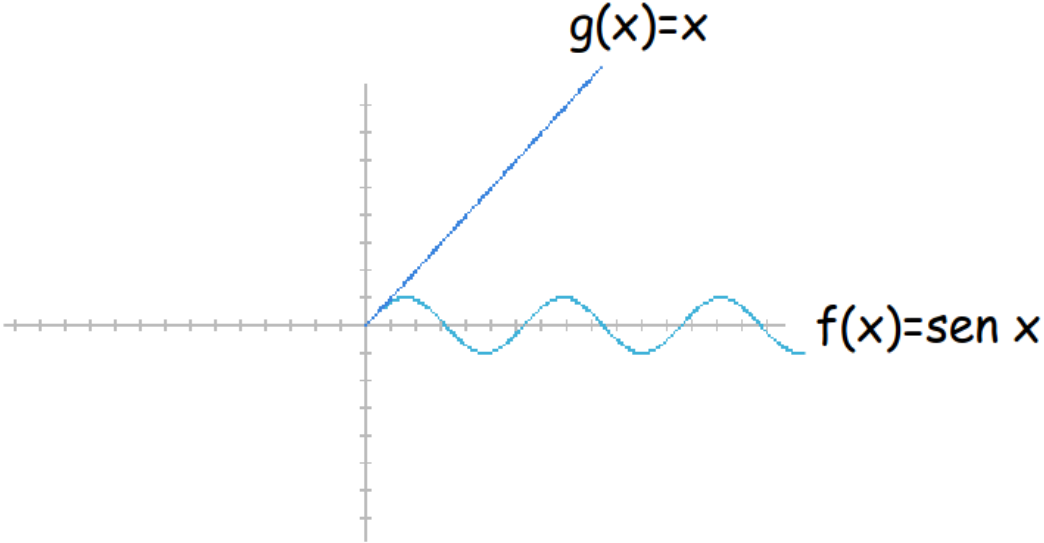


Siempre se cumple que f(x)<g(x) para cualquier valor x

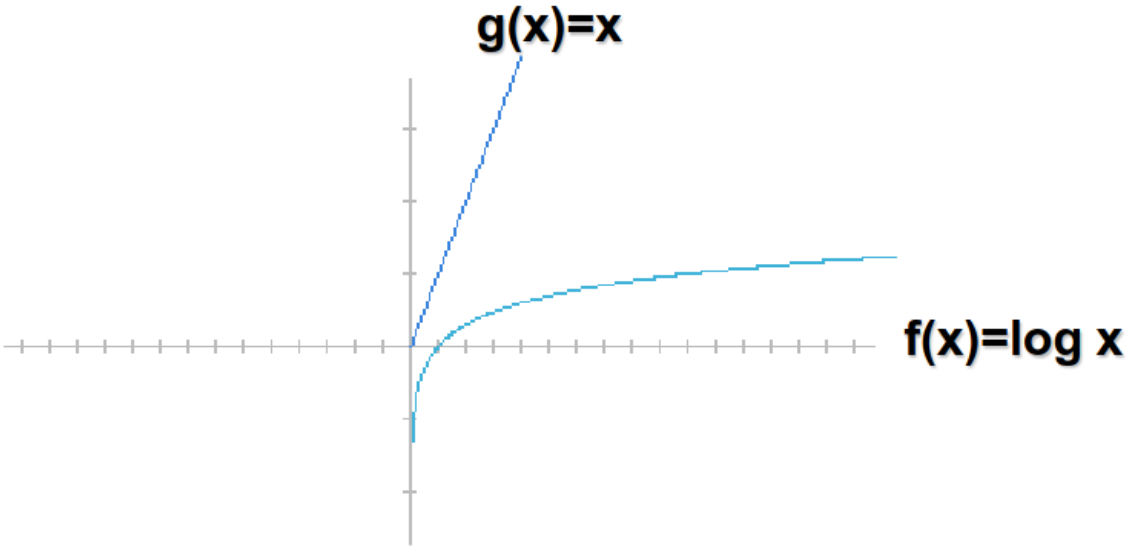
g(x) es una cota superior de f(x)

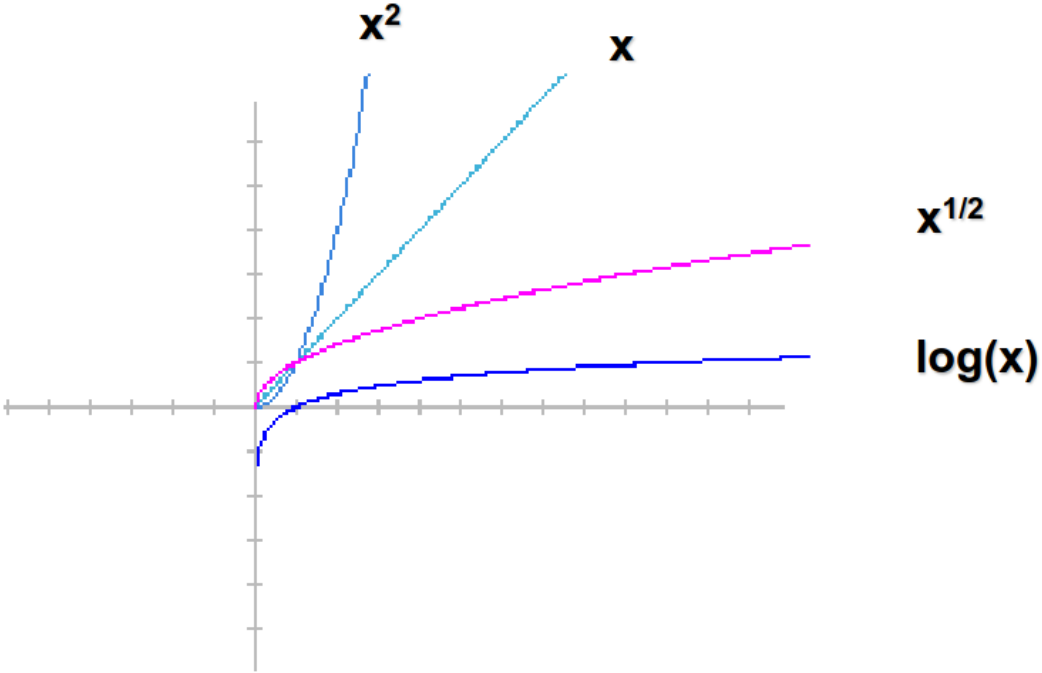

g(x)=x²
f(x)=x

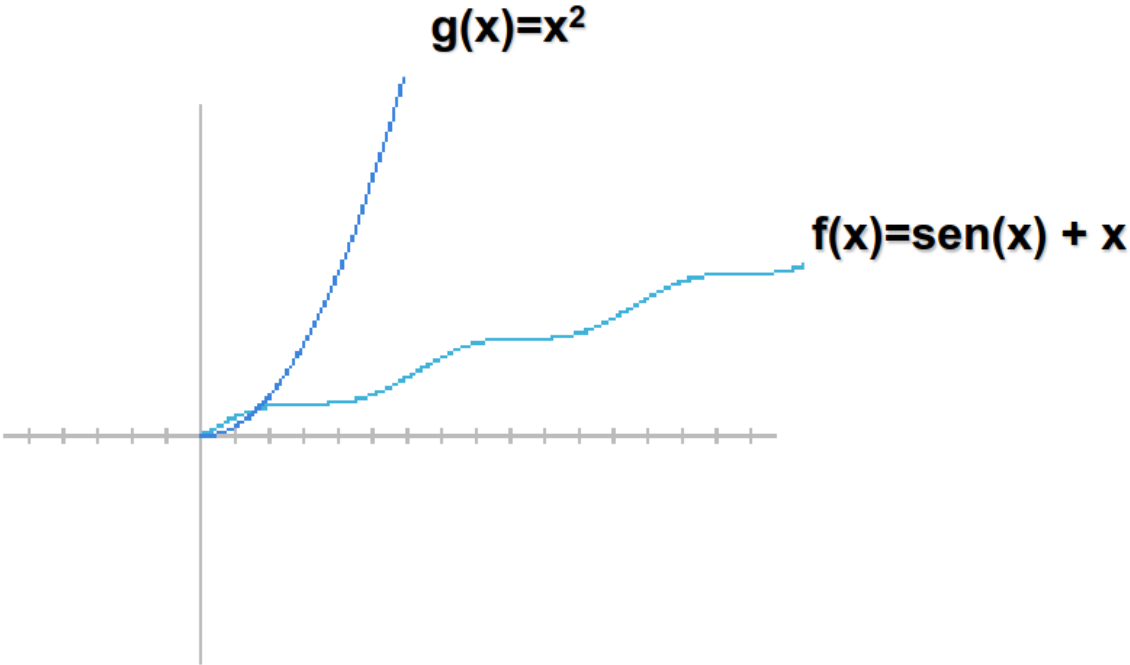

g(x)=x
f(x)=x^1/2



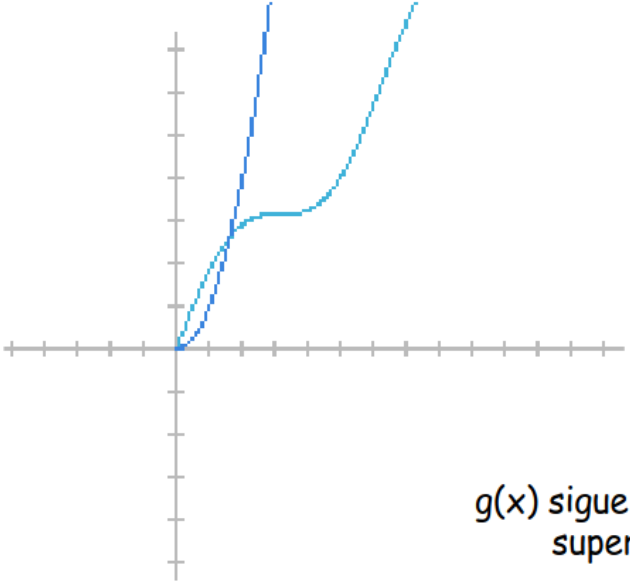
¿Está (sen x) por debajo de x^2?


g(x)=x
f(x)=log x


x^2
x
$x^{1/2}$
$\log(x)$

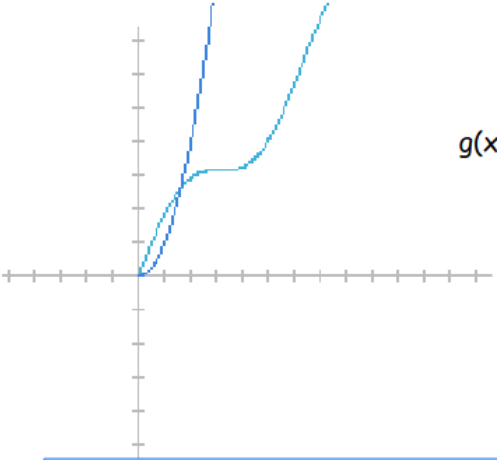

g(x)=x²
f(x)=sen(x) + x

$g(x)=x^2$ $f(x)=sen(x) + x$



g(x) sigue siendo una cota superior de f(x)

$g(x)=x^2$ $f(x)=sen(x) + x$

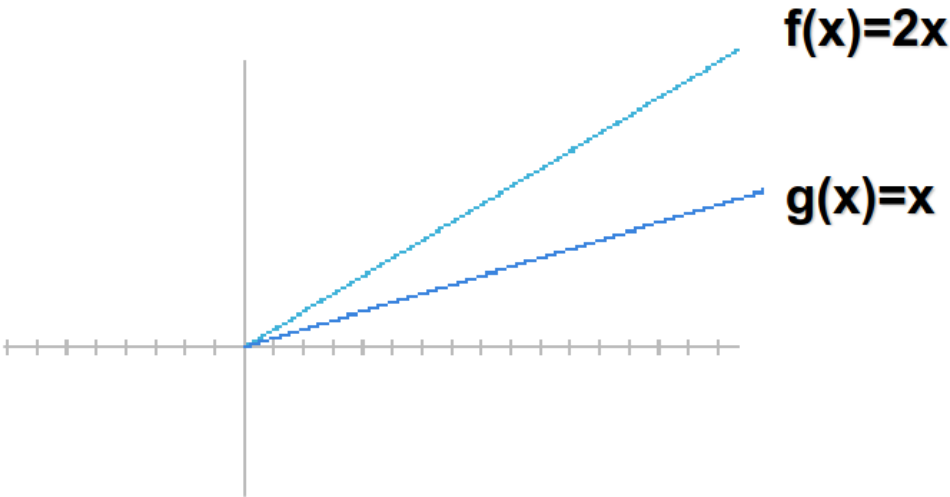


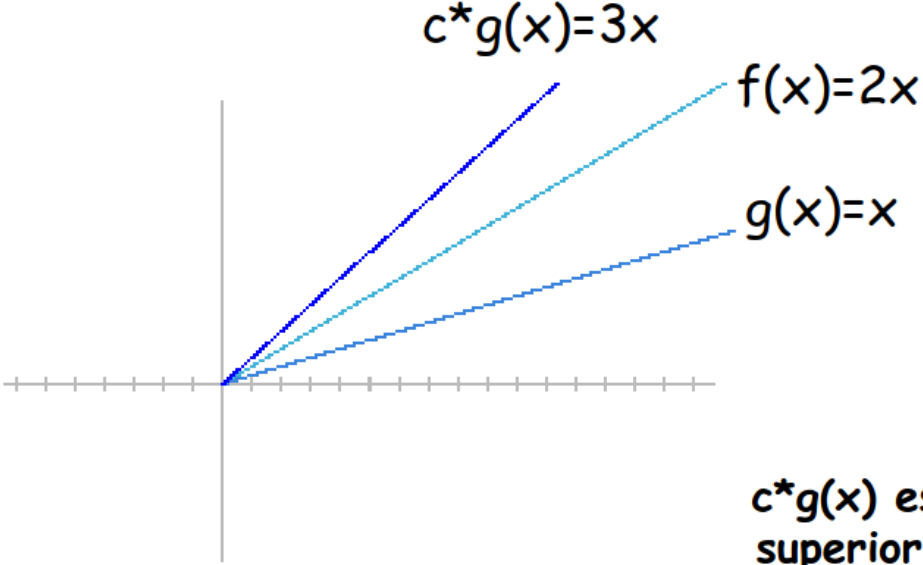
g(x) sigue siendo una cota superior de f(x)

Para que una función sea cota superior de otra, debe serlo para <u>todos</u> los valores de **x≥k**

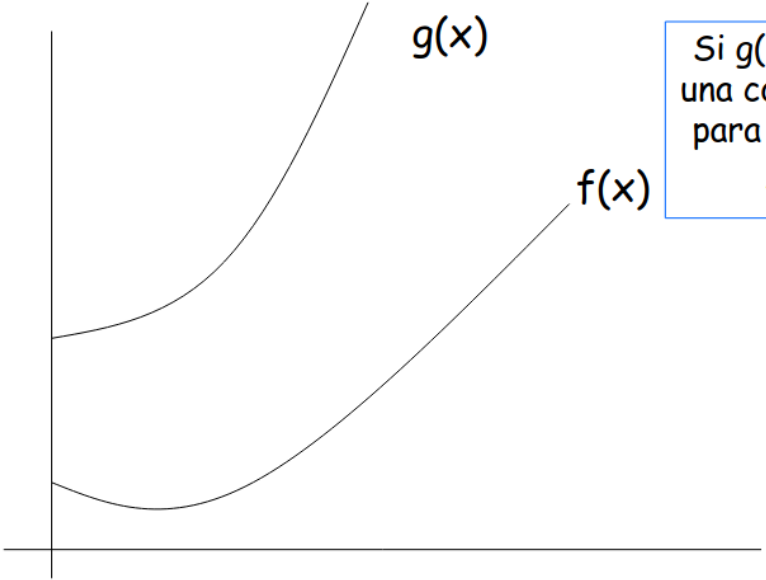
Pueden existir valores de x<k para los cuales no sea cota superior

Buscar una cota superior para f(x) en términos de g(x)



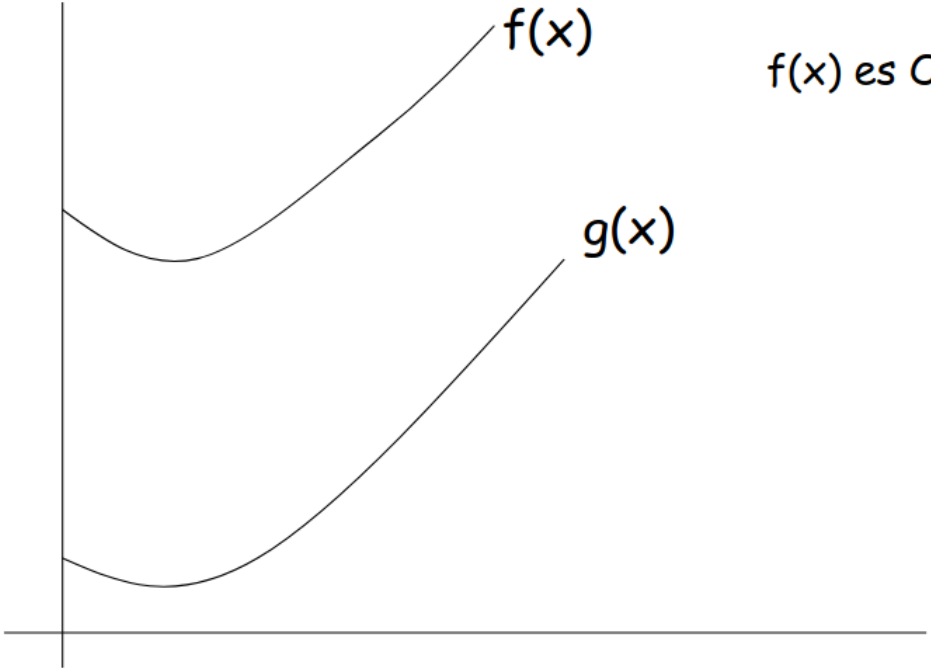


c*g(x) es una cota superior para f(x)

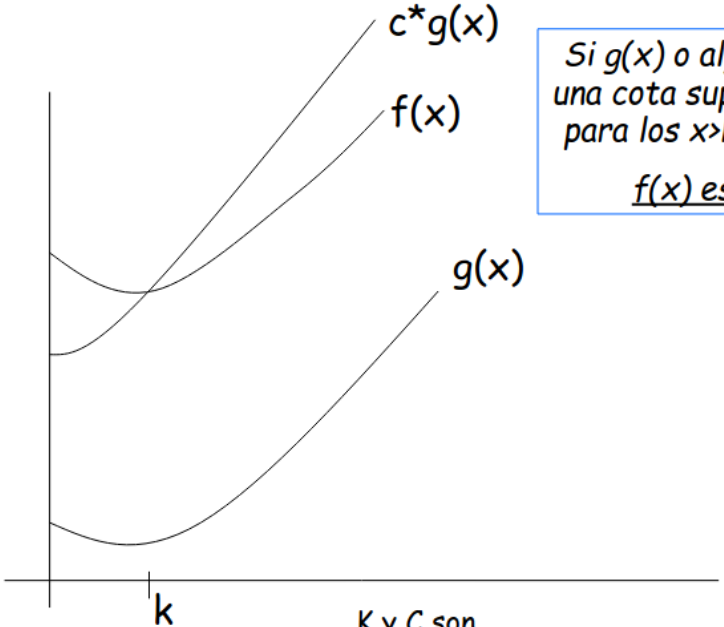


Si g(x) o algún c*g(x) es una cota superior de f(x), para los x>k, se dice que

f(x) es O(g(x))



f(x) es O(g(x)) ?



K y C son constantes

*Si g(x) o algún c*g(x) es una cota superior de f(x), para los x>k, se dice que*

<u>f(x) es O(g(x))</u>

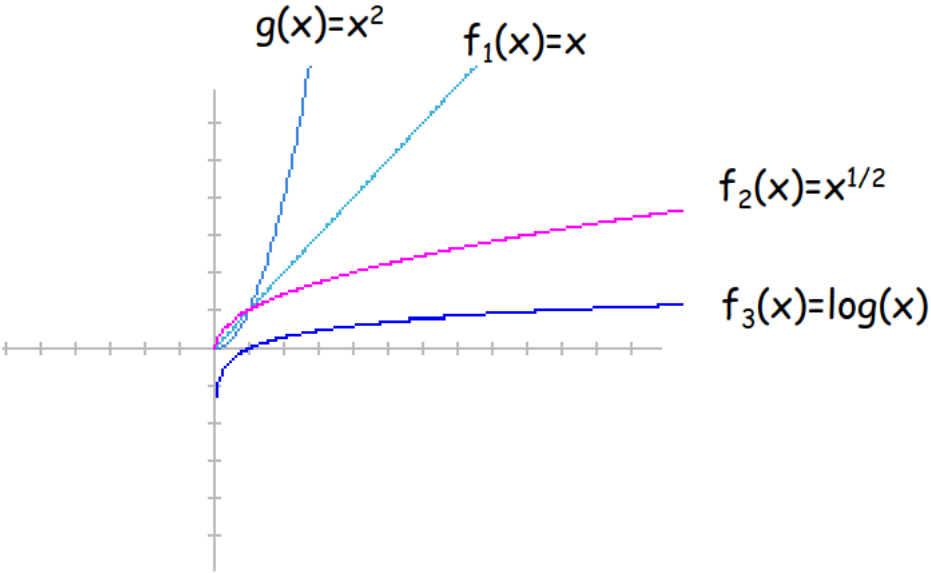
Crecimiento de funciones

Dadas dos funciones f y g, se dice que $f(x)$ es $O(g(x))$ si existen constantes C y k tales que:

$$f(x) \leq C \cdot g(x)$$

se cumple para todos los $x \geq k$

Ψ *Donald Knuth (1938 -) - Símbolo de Landau*



g(x) es una cota superior de
$f_1(x)$, $f_2(x)$, $f_3(x)$

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Formalmente la notación O representa un **conjunto de funciones**

$O(g(x)) = \{f(x) \mid$ existen constantes positivas c y k,
tales que $0 \leq f(x) \leq c \cdot g(x)$, para todos los $x \geq k\}$

En otras palabras: $O(g(x))$ es el conjunto de **todas las funciones que son acotadas superiormente por** $c \cdot g(x)$ a partir de algún punto k.

- Cuando decimos $\mathbf{f(x) = O(g(x))}$, en realidad significa $\mathbf{f(x) \in O(g(x))}$: f **pertenece** al conjunto de funciones que g acota por arriba (salvo constante).
- La función $g(x)$ actúa como **cota superior asintótica**: para $x \geq k$, $f(x)$ nunca supera a $c \cdot g(x)$, sin importar cuán grande sea x.

Crecimiento de funciones

Por ejemplo, todas estas funciones pertenecen a $O(x^2)$:

$$O(x^2) \ni \quad x^2, \quad x^2 + 5x + 3, \quad x, \quad x^{1/2}, \quad \log(x), \quad \ldots$$

- $x^2 \in O(x^2)$ porque $x^2 \leq 1 \cdot x^2$ para $x \geq 1$
- $x \in O(x^2)$ porque $x \leq 1 \cdot x^2$ para $x \geq 1$
- $\log(x) \in O(x^2)$ porque $\log(x) \leq 1 \cdot x^2$ para $x \geq 1$

En cambio, $x^3 \notin O(x^2)$ porque ningún c y k logran que $x^3 \leq c \cdot x^2$ para todo x suficientemente grande.

Crecimiento de funciones

Muestre que $5x = O(x^2)$

Estrategia: elegir $C = 1$ y **despejar** la condición sobre x.

Demostración algebraica:

Queremos encontrar k tal que para $x \geq k$:

$$5x \leq 1 \cdot x^2$$

Dividiendo ambos lados por $x > 0$:

$$5 \leq x$$

La desigualdad se cumple exactamente cuando $x \geq 5$.

Por lo tanto, tomando $C = 1$ y $k = 5$:

$$5x \leq x^2 \quad \forall x \geq 5 \quad \blacksquare$$

Verificación:

x	x^2	$5x$	
3	9	15	✗
4	16	20	✗
5	25	25	✓
6	36	30	✓

La tabla *confirma* el resultado algebraico: el cruce ocurre en $x = 5$.

Crecimiento de funciones

Muestre que $\mathbf{2x + 3 = O(x)}$

Estrategia: Elegir $C = 3$ para que $3x$ absorba el coeficiente 2 de x y la constante $+3$.

Demostración algebraica:

Necesitamos encontrar k tal que:

$$2x + 3 \leq 3x$$

Restando $2x$ de ambos lados:

$$3 \leq x$$

La desigualdad se cumple cuando $x \geq 3$.

Conclusión: $C = 3$ y $k = 3$

$$2x + 3 \leq 3x \quad \forall x \geq 3 \quad \blacksquare$$

Verificación:

x	$3x$	$2x + 3$	
1	3	5	✗
2	6	7	✗
3	9	9	✓
4	12	11	✓

El margen $(3 - 2)x = x$ absorbe la constante $+3$ cuando $x \geq 3$.

Crecimiento de funciones

Muestre que $7x^2 = O(x^3)$

Estrategia: elegir $C = 1$ y **despejar** la condición sobre x.

Demostración algebraica:

Queremos encontrar k tal que para $x \geq k$:

$$7x^2 \leq 1 \cdot x^3$$

Dividiendo ambos lados por $x^2 > 0$:

$$7 \leq x$$

La desigualdad se cumple exactamente cuando $x \geq 7$.

Por lo tanto, tomando $C = 1$ y $k = 7$:

$$7x^2 \leq x^3 \quad \forall x \geq 7 \quad \blacksquare$$

Verificación:

x	x^3	$7x^2$	
5	125	175	✗
6	216	252	✗
7	343	343	✓
8	512	448	✓

La tabla *confirma* el resultado algebraico: el cruce ocurre en $x = 7$.

Crecimiento de funciones

Es x^3, $O(7x^2)$

Crecimiento de funciones

Es x^3, $O(7x^2)$

$x^3 < c\ 7x^2$

$x < 7c$

Ya que no se cumple para todos los $x > k$, no es cierto que x^3 sea $O(7x^2)$

Crecimiento de funciones

Ejercicios

Demostrar que:

- $x^2 + 2x + 1$ es $O(x^2)$
- $\log n$ es $O(n)$, parta sabiendo que $n < 2^n$
- $x^2 + 4x + 17$ es $O(x^3)$ pero que x^3 no es $O(x^2 + 4x + 17)$
- $x \log x$ es $O(x^2)$

Consulta

- Explique qué significa que una función sea $\Omega(1)$
- Explique qué significa que una función sea $O(1)$

Crecimiento de funciones

Cota superior asintótica

Cuando se determina que el tiempo de computo de un algoritmo es $O(g(x))$ se establece una cota superior

Dentro del análisis para obtener la cota superior, se debe considerar el peor caso, de esta forma se consigue tener una cota que no puede resultar peor

Crecimiento de funciones

Cota superior asintótica

Suponga que para un algoritmo se encontró que $T_1(n) = O(n^2)$ y para otro que $T_2(n) = O(n \lg n)$, qué representan estos resultados?

¿Qué se puede esperar en cuanto al tiempo de ejecución de los algoritmos?

¿Se puede asegurar que siempre es mejor el algoritmo 2 que el 1?

Crecimiento de funciones

Cota superior asintótica

Es un abuso decir que el tiempo de ejecución del insertion sort es $O(n^2)$, esto se puede asegurar, para el peor caso.

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

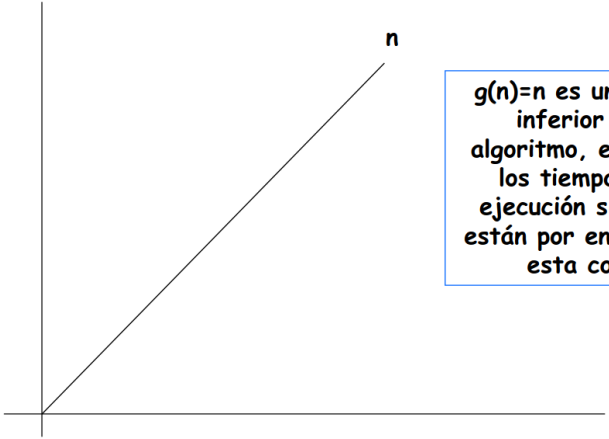
Cota inferior

$\Omega(g(x)) = \{\, f(x) \mid$ existen constantes positivas c y k, tales que $0 \leq c \cdot g(x) \leq f(x)$, para todos los $x \geq k \,\}$

Crecimiento de funciones

Cota inferior

Suponga que para un algoritmo se encontró que $T_1(n) = \Omega(n)$



g(n)=n es una cota inferior del algoritmo, esto es, los tiempos de ejecución siempre están por encima de esta cota

Crecimiento de funciones

Cota inferior

Suponga que para un algoritmo se encontró que $T_1(n) = \Omega(n^2)$ y para otro que $T_2(n) = \Omega(n \lg n)$, qué representan estos resultados?

¿Qué se puede esperar en cuanto al tiempo de ejecución de los algoritmos?

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Notación $\Theta(f(n))$

$f(n) = \Theta(g(n))$ sii $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Complejidades de los algoritmos

Complejidad	Terminología
$O(1)$	Complejidad constante
$O(\log n)$	Complejidad logarítmica
$O(n)$	Complejidad lineal
$O(n \log n)$	Complejidad $n \log n$
$O(n^b)$	Complejidad polinomial
$O(b^n)$	Complejidad exponencial
$O(n!)$	Complejidad factorial

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Crecimiento de funciones

Complejidades de los algoritmos

Un problema que se puede resolver utilizando un algoritmo con complejidad polinómica en el peor caso se llama tratable. De no se así se llama intratable

Un problema para el cual no existe un programa que lo pueda solucionar se llama irresoluble. De no ser así se llaman resolubles. (Halting problem - Turing)

Ψ Alan Turing (1912–1954) Problema de la decisión de Hilbert

Crecimiento de funciones

Complejidades de los algoritmos

Existen problemas que no se pueden resolver, en el peor caso en tiempo polinomial, pero que dada alguna solución, se puede comprobar que es efectivamente una solución. Estos problemas se llaman NP (Polinómico - No determinista)

Existe un conjunto de problemas NP que además, si se llegase a encontrar una solución en tiempo polinómico, daría solución a un conjunto de problemas relacionados. Este tipo de problema se conoce como NP-completos

Crecimiento de funciones

Complejidades de los algoritmos

El problema de la satisfactibilidad es un problema NP-completo.

Dada una asignación de valores de verdad, se puede verificar en tiempo polinómico si tal asignación satisface, o no, una fórmula proposicional. Sin embargo, no existe un algoritmo que en tiempo polinómico pueda encontrar la asignación de valores de verdad para que una fórmula cualquiera se satisfaga.

Contenido

1. Crecimiento de funciones

2. Notación O

3. Notación Ω

4. Notación Θ

5. Terminología de complejidades

6. Clasificación de problemas

7. Análisis de algoritmos

Análisis de algoritmos

Meta: Comparar algoritmos que resuelven un mismo problema

- Correctitud
- Eficiencia
  - Tiempo
  - Espacio
- Estructuras de datos utilizadas
- Modelo computacional
- El tipo y número de datos con los cuales trabaja (escalabilidad)

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	
2	**while** $i \leq$ len(mat1)	c_2	
3	$\quad j \leftarrow 1$	c_3	
4	$\quad$ **while** $j \leq$ len(mat2)	c_4	
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	
6	$\quad\quad j \leftarrow j + 1$	c_6	
7	$\quad i \leftarrow i + 1$	c_7	

Supuesto: len(mat1) $=$ len(mat2) $= n$ (matrices cuadradas de la misma dimensión). Además, en nuestro pseudocódigo asumimos indexación de arreglos y matrices desde 1.

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	
3	$\quad j \leftarrow 1$	c_3	
4	$\quad$ **while** $j \leq$ len(mat2)	c_4	
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	
6	$\quad\quad j \leftarrow j + 1$	c_6	
7	$\quad i \leftarrow i + 1$	c_7	

Línea 1: inicialización, se ejecuta exactamente **1** vez.

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	$n+1$
3	$\quad j \leftarrow 1$	c_3	
4	$\quad$ **while** $j \leq$ len(mat2)	c_4	
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	
6	$\quad\quad j \leftarrow j+1$	c_6	
7	$\quad i \leftarrow i+1$	c_7	

Línea 2: el `while` externo evalúa $n+1$ veces (n verdaderas $+1$ falsa al salir).

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	$n+1$
3	$\quad j \leftarrow 1$	c_3	n
4	$\quad$ **while** $j \leq$ len(mat2)	c_4	
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	
6	$\quad\quad j \leftarrow j+1$	c_6	
7	$\quad i \leftarrow i+1$	c_7	

Línea 3: j se reinicia en cada iteración del ciclo externo $\Rightarrow$ n veces.

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	$n+1$
3	$\quad j \leftarrow 1$	c_3	n
4	$\quad$ **while** $j \leq$ len(mat2)	c_4	$n(n+1)$
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	
6	$\quad\quad j \leftarrow j+1$	c_6	
7	$\quad i \leftarrow i+1$	c_7	

Línea 4: el `while` interno evalúa $n+1$ veces por cada iteración externa $\Rightarrow n(n+1)$.

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	$n+1$
3	$j \leftarrow 1$	c_3	n
4	**while** $j \leq$ len(mat2)	c_4	$n(n+1)$
5	$mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	n^2
6	$j \leftarrow j+1$	c_6	
7	$i \leftarrow i+1$	c_7	

Línea 5: el cuerpo interno ejecuta n veces por iteración externa $\Rightarrow n^2$.

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	$n+1$
3	$j \leftarrow 1$	c_3	n
4	**while** $j \leq$ len(mat2)	c_4	$n(n+1)$
5	$mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	n^2
6	$j \leftarrow j+1$	c_6	n^2
7	$i \leftarrow i+1$	c_7	

Línea 6: igual al cuerpo interno $\Rightarrow n^2$.

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	$n+1$
3	$\quad j \leftarrow 1$	c_3	n
4	$\quad$ **while** $j \leq$ len(mat2)	c_4	$n(n+1)$
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	n^2
6	$\quad\quad j \leftarrow j+1$	c_6	n^2
7	$\quad i \leftarrow i+1$	c_7	n

Línea 7: i avanza una vez por iteración externa $\Rightarrow$ n veces.

Análisis de algoritmos

#	Instrucción	Costo	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq$ len(mat1)	c_2	$n+1$
3	$\quad j \leftarrow 1$	c_3	n
4	$\quad$ **while** $j \leq$ len(mat2)	c_4	$n(n+1)$
5	$\quad\quad mat3[i][j] \leftarrow mat1[i][j] + mat2[i][j]$	c_5	n^2
6	$\quad\quad j \leftarrow j+1$	c_6	n^2
7	$\quad i \leftarrow i+1$	c_7	n

Costo total (sumando cada línea):

$$T(n) = 1 + (n+1) + n + n(n+1) + n^2 + n^2 + n$$

$$= 1 + n + 1 + n + n^2 + n + n^2 + n^2 + n$$

$$= 3n^2 + 4n + 2$$

$$\Rightarrow \quad T(n) = O(n^2)$$

Análisis de algoritmos — Enfoque por sumatorias

#	Instrucción	Cto.	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq n$	c_2	$n+1$
3	$j \leftarrow 1$	c_3	**?**
4	**while** $j \leq n$	c_4	**?**
5	$mat3[i][j] \leftarrow \cdots$	c_5	**?**
6	$j \leftarrow j+1$	c_6	**?**
7	$i \leftarrow i+1$	c_7	n

Supuesto: len(mat1) $=$ len(mat2) $= n$.

Líneas 1, 2 y 7: análisis directo (igual que antes).

Zoom en líneas 3–6: están dentro del ciclo externo y contienen un ciclo interno. Hay que analizar cuántas veces ejecuta el ciclo interno para **cada valor válido de** i.

Análisis de algoritmos — Enfoque por sumatorias

#	Instrucción	Cto.	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq n$	c_2	$n+1$
3	$j \leftarrow 1$	c_3	**?**
4	**while** $j \leq n$	c_4	**?**
5	$mat3[i][j] \leftarrow \cdots$	c_5	**?**
6	$j \leftarrow j+1$	c_6	**?**
7	$i \leftarrow i+1$	c_7	n

Línea 3: $j \leftarrow 1$ se ejecuta en cada iteración del ciclo externo.

Como i toma n valores válidos $(1, 2, \ldots, n)$:

$$\text{Línea } 3 = n \text{ veces}$$

Análisis de algoritmos — Enfoque por sumatorias

#	Instrucción	Cto.	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq n$	c_2	$n+1$
3	$j \leftarrow 1$	c_3	n
4	**while** $j \leq n$	c_4	**?**
5	$mat3[i][j] \leftarrow \cdots$	c_5	**?**
6	$j \leftarrow j+1$	c_6	**?**
7	$i \leftarrow i+1$	c_7	n

Definimos t_i = veces que el `while` interno (línea 4) evalúa su condición cuando i tiene el valor i.

Para **cualquier** valor válido de i, j recorre:

$$j = 1,\ 2,\ 3,\ \ldots,\ n,\ \underbrace{n+1}_{\text{falso, sale}}$$

$\Rightarrow t_i = n+1$ es **constante** para todo $i \in \{1, \ldots, n\}$.

Análisis de algoritmos — Enfoque por sumatorias

#	Instrucción	Cto.	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq n$	c_2	$n+1$
3	$j \leftarrow 1$	c_3	n
4	**while** $j \leq n$	c_4	$n(n+1)$
5	$mat3[i][j] \leftarrow \cdots$	c_5	**?**
6	$j \leftarrow j+1$	c_6	**?**
7	$i \leftarrow i+1$	c_7	n

Línea 4 — sumamos t_i sobre todos los valores válidos de i:

$$\sum_{i=1}^{n} t_i = \sum_{i=1}^{n} (n+1) = n \cdot (n+1)$$

Análisis de algoritmos — Enfoque por sumatorias

#	Instrucción	Cto.	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq n$	c_2	$n+1$
3	$j \leftarrow 1$	c_3	n
4	**while** $j \leq n$	c_4	$n(n+1)$
5	$mat3[i][j] \leftarrow \cdots$	c_5	n^2
6	$j \leftarrow j+1$	c_6	n^2
7	$i \leftarrow i+1$	c_7	n

Líneas 5 y 6 solo ejecutan cuando la condición de línea 4 es **verdadera**: $t_i - 1$ veces por cada i (se resta la evaluación falsa).

$$\sum_{i=1}^{n}(t_i - 1) = \sum_{i=1}^{n}(n+1-1) = \sum_{i=1}^{n} n = n^2$$

Análisis de algoritmos — Enfoque por sumatorias

#	Instrucción	Cto.	Veces
1	$i \leftarrow 1$	c_1	1
2	**while** $i \leq n$	c_2	$n+1$
3	$j \leftarrow 1$	c_3	n
4	**while** $j \leq n$	c_4	$n(n+1)$
5	$mat3[i][j] \leftarrow \cdots$	c_5	n^2
6	$j \leftarrow j+1$	c_6	n^2
7	$i \leftarrow i+1$	c_7	n

Costo total (mismo resultado, deducido por sumatorias):

$$T(n) = 1 + (n+1) + n + n(n+1) + n^2 + n^2 + n$$

$$= 3n^2 + 4n + 2 \;\Rightarrow\; T(n) = O(n^2)$$

Análisis de algoritmos — Ejemplo 2

#	Instrucción	Cto.	Veces
1	$s \leftarrow 0$	c_1	
2	$i \leftarrow 1$	c_2	
3	**while** $i \leq n$	c_3	
4	$t \leftarrow 0$	c_4	
5	$j \leftarrow 1$	c_5	
6	**while** $j \leq i$	c_6	**?**
7	$t \leftarrow t + 1$	c_7	**?**
8	$j \leftarrow j + 1$	c_8	**?**
9	$s \leftarrow s + t$	c_9	
10	$i \leftarrow i + 1$	c_{10}	

Supuesto: el ciclo externo depende de n.

Analizaremos primero las líneas cuyo conteo es directo.

Líneas 6, 7 y 8 requieren análisis con **sumatorias**: su ciclo interno depende de i, no de n.

Análisis de algoritmos — Ejemplo 2

#	Instrucción	Cto.	Veces
1	$s \leftarrow 0$	c_1	1
2	$i \leftarrow 1$	c_2	1
3	**while** $i \leq n$	c_3	
4	$t \leftarrow 0$	c_4	
5	$j \leftarrow 1$	c_5	
6	**while** $j \leq i$	c_6	**?**
7	$t \leftarrow t + 1$	c_7	**?**
8	$j \leftarrow j + 1$	c_8	**?**
9	$s \leftarrow s + t$	c_9	
10	$i \leftarrow i + 1$	c_{10}	

Líneas 1 y 2: inicializaciones simples, antes del ciclo.

Se ejecutan exactamente **1 sola vez** cada una.

Análisis de algoritmos — Ejemplo 2

#	Instrucción	Cto.	Veces
1	$s \leftarrow 0$	c_1	1
2	$i \leftarrow 1$	c_2	1
3	**while** $i \leq n$	c_3	$n+1$
4	$t \leftarrow 0$	c_4	
5	$j \leftarrow 1$	c_5	
6	**while** $j \leq i$	c_6	**?**
7	$t \leftarrow t+1$	c_7	**?**
8	$j \leftarrow j+1$	c_8	**?**
9	$s \leftarrow s+t$	c_9	
10	$i \leftarrow i+1$	c_{10}	

Línea 3: condición del `while` externo.

i toma valores válidos $1, 2, \ldots, n$ y en $i = n+1$ la condición es falsa y se sale.

$$\Rightarrow \; n+1 \text{ evaluaciones}$$

Análisis de algoritmos — Ejemplo 2

#	Instrucción	Cto.	Veces
1	$s \leftarrow 0$	c_1	1
2	$i \leftarrow 1$	c_2	1
3	**while** $i \leq n$	c_3	$n+1$
4	$t \leftarrow 0$	c_4	n
5	$j \leftarrow 1$	c_5	n
6	**while** $j \leq i$	c_6	**?**
7	$t \leftarrow t+1$	c_7	**?**
8	$j \leftarrow j+1$	c_8	**?**
9	$s \leftarrow s+t$	c_9	
10	$i \leftarrow i+1$	c_{10}	

Líneas 4 y 5: al nivel del ciclo externo.

Solo se ejecutan en las n iteraciones válidas del `while` externo.

$\Rightarrow$ n veces cada una

Análisis de algoritmos — Ejemplo 2

#	Instrucción	Cto.	Veces
1	$s \leftarrow 0$	c_1	1
2	$i \leftarrow 1$	c_2	1
3	**while** $i \leq n$	c_3	$n+1$
4	$t \leftarrow 0$	c_4	n
5	$j \leftarrow 1$	c_5	n
6	**while** $j \leq i$	c_6	**?**
7	$t \leftarrow t+1$	c_7	**?**
8	$j \leftarrow j+1$	c_8	**?**
9	$s \leftarrow s+t$	c_9	n
10	$i \leftarrow i+1$	c_{10}	n

Líneas 9 y 10: también al nivel del ciclo externo (igual que 4 y 5).

Se ejecutan en las n iteraciones válidas.

$\Rightarrow$ n veces cada una

Análisis de algoritmos — Ejemplo 2

#	Instrucción	Cto.	Veces
1	$s \leftarrow 0$	c_1	1
2	$i \leftarrow 1$	c_2	1
3	**while** $i \leq n$	c_3	$n+1$
4	$t \leftarrow 0$	c_4	n
5	$j \leftarrow 1$	c_5	n
6	**while** $j \leq i$	c_6	zoom
7	$t \leftarrow t+1$	c_7	zoom
8	$j \leftarrow j+1$	c_8	zoom
9	$s \leftarrow s+t$	c_9	n
10	$i \leftarrow i+1$	c_{10}	n

Zoom en líneas 6, 7 y 8:

El `while` interno tiene la condición $j \leq i$.

El límite cambia con cada iteración del ciclo externo: **cuando** $i = 1$ **va hasta 1, cuando** $i = 2$ **va hasta 2, …**

$\Rightarrow$ El número de veces no es constante: usaremos **sumatorias** en el siguiente slide.

Análisis — Zoom en líneas 6, 7 y 8

Definimos t_i = veces que la **línea 6** evalúa su condición cuando el contador externo vale i.

Ensayemos caso a caso:

- $i = 1$: j toma $1,\ \mathbf{2}$. En $j = 1$: válido. En $j = 2$: **rompe**.

$$t_1 = 2 = 1 + 1$$

- $i = 2$: j toma $1, 2,\ \mathbf{3}$. En $j = 1, 2$: válido. En $j = 3$: **rompe**.

$$t_2 = 3 = 2 + 1$$

- $i = 3$: j toma $1, 2, 3,\ \mathbf{4}$. En $j = 1, 2, 3$: válido. En $j = 4$: **rompe**.

$$t_3 = 4 = 3 + 1$$

i	j válidos	t_i	Líneas 7, 8
1	1	2	1 vez
2	$1,\ 2$	3	2 veces
3	$1,\ 2,\ 3$	4	3 veces
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	$1, \ldots, n$	$n+1$	n veces
Total	—	$\frac{n(n+3)}{2}$	$\frac{n(n+1)}{2}$

Fórmula de Gauss usada:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Análisis — Zoom en líneas 6, 7 y 8

Definimos t_i = veces que la **línea 6** evalúa su condición cuando el contador externo vale i.

Patrón general:

$$t_i = i + 1$$

Siempre i valores válidos + 1 evaluación de ruptura.

Línea 6 — sumatoria sobre todos los i válidos:

$$\sum_{i=1}^{n}(i+1) = \sum_{i=1}^{n} i + \sum_{i=1}^{n} 1$$

$$= \frac{n(n+1)}{2} + n$$

$$= \frac{n(n+3)}{2}$$

i	j válidos	t_i	Líneas 7, 8
1	1	2	1 vez
2	$1,\ 2$	3	2 veces
3	$1,\ 2,\ 3$	4	3 veces
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	$1, \ldots, n$	$n+1$	n veces
Total	—	$\frac{n(n+3)}{2}$	$\frac{n(n+1)}{2}$

Fórmula de Gauss usada:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Análisis — Zoom en líneas 6, 7 y 8

Definimos t_i = veces que la **línea 6** evalúa su condición cuando el contador externo vale i.

Líneas 7 y 8 solo ejecutan en las $t_i - 1$ iteraciones *válidas* del ciclo interno (sin contar la evaluación de ruptura):

$$t_i - 1 = (i+1) - 1 = i$$

Por tanto, **cada una** de las dos líneas ejecuta:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

i	j válidos	t_i	Líneas 7, 8
1	1	2	1 vez
2	$1,\ 2$	3	2 veces
3	$1,\ 2,\ 3$	4	3 veces
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	$1, \ldots, n$	$n+1$	n veces
Total	—	$\frac{n(n+3)}{2}$	$\frac{n(n+1)}{2}$

Fórmula de Gauss usada:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Análisis — Costo total

Línea(s)	Descripción	Veces
1, 2	Inicializaciones	$1 + 1$
3	`while` externo	$n + 1$
4, 5	Inicializaciones internas	$n + n$
6	`while` interno	$\frac{n(n+3)}{2}$
7, 8	Cuerpo del ciclo interno	$\frac{n(n+1)}{2} + \frac{n(n+1)}{2}$
9, 10	Acumuladores externos	$n + n$

Análisis — Costo total

Línea(s)	Descripción	Veces
1, 2	Inicializaciones	$1 + 1$
3	`while` externo	$n + 1$
4, 5	Inicializaciones internas	$n + n$
6	`while` interno	$\frac{n(n+3)}{2}$
7, 8	Cuerpo del ciclo interno	$\frac{n(n+1)}{2} + \frac{n(n+1)}{2}$
9, 10	Acumuladores externos	$n + n$

Sumando todas las líneas:

$$T(n) = 2 + (n+1) + 2n + \frac{n(n+3)}{2} + n(n+1) + 2n$$

$$= 3 + 5n + \frac{n^2 + 3n}{2} + n^2 + n = 3 + 6n + \frac{n^2 + 3n}{2} + n^2$$

Análisis — Costo total

Línea(s)	Descripción	Veces
1, 2	Inicializaciones	$1+1$
3	`while` externo	$n+1$
4, 5	Inicializaciones internas	$n+n$
6	`while` interno	$\frac{n(n+3)}{2}$
7, 8	Cuerpo del ciclo interno	$\frac{n(n+1)}{2}+\frac{n(n+1)}{2}$
9, 10	Acumuladores externos	$n+n$

Sumando todas las líneas:

$$T(n) = 2 + (n+1) + 2n + \frac{n(n+3)}{2} + n(n+1) + 2n$$

$$= 3 + 5n + \frac{n^2+3n}{2} + n^2 + n = 3 + 6n + \frac{n^2+3n}{2} + n^2$$

$$= \frac{6 + 12n + n^2 + 3n + 2n^2}{2} = \frac{3n^2 + 15n + 6}{2}$$

$$\Rightarrow \boxed{T(n) = O(n^2)}$$